Data Preprocessing & Feature Engineering

Theory Concepts Reference Document

Holistic Data Preparer — Final Project

Customer Credit Risk Dataset | FinTech ML Pipeline

1. Data Analysis

1.1 What is Data Analysis?

Definition: Data Analysis is the systematic process of collecting, cleaning, transforming, and modelling data to extract meaningful insights, support decisions, and identify patterns.

Data analysis spans four maturity levels:

Descriptive: What happened? (summary statistics, aggregations)

Diagnostic: Why did it happen? (correlation, root-cause)

Predictive: What will happen? (ML models, forecasting)

Prescriptive: What should we do? (optimization, recommendations)

1.2 How to Plan a Data Science Project

A structured project plan follows the CRISP-DM framework:

Step	Description
1. Business Understanding	Define the problem, success metrics, and stakeholder requirements.
2. Data Understanding	Explore data sources, understand features, assess quality.
3. Data Preparation	Clean, impute, encode, scale, and engineer features.
4. Modelling	Select algorithms, train models, tune hyperparameters.
5. Evaluation	Assess model performance against business KPIs.
6. Deployment	Productionize, monitor, and retrain as needed.

2. Tensors & NumPy

Definition: A Tensor is a generalization of scalars, vectors, and matrices to arbitrary dimensions (axes/rank). In ML, data is always represented as tensors.

Tensor Ranks:

Rank	Name	Shape	Example
0	Scalar	()	42, 3.14
1	Vector	(n,)	[1,2,3,4]
2	Matrix	(m,n)	DataFrame rows×cols
3	3-D Tensor	(d,m,n)	Image: (batch, H, W)
4	4-D Tensor	(b,d,m,n)	Video: (batch, frames, H, W)

```
import numpy as np scalar = np.array(42) # rank-0 vector = np.array([1,2,3]) # rank-1, shape (3,) matrix = np.arange(6).reshape(2,3) # rank-2, shape (2,3) tensor = np.random.rand(4,28,28) # rank-3 (batch of images)
```

3. Missing Value Imputation

Definition: Missing Value Imputation is the process of replacing absent data points with estimated values so that the dataset is complete for analysis and modelling.

3.1 Types of Missingness

MCAR (Missing Completely at Random): Absence is independent of data — safe to ignore or impute with simple methods.

MAR (Missing at Random): Absence depends on observed variables — use model-based imputation.

MNAR (Missing Not at Random): Absence depends on the missing value itself — create a missing indicator.

3.2 Imputation Methods

Method	scikit-learn Class	Use Case	Applied To
Mean Imputation	SimpleImputer(strategy='mean')	Symmetric numerical	age
Median Imputation	SimpleImputer(strategy='median')	Skewed numerical	annual_income
Most Frequent	SimpleImputer(strategy='most_frequent')	Categorical	gender, employment_type
Constant Fill	SimpleImputer(strategy='constant')	Known default value	—
Random Sample	Custom	Preserves distribution	annual_income
KNN Imputer	KNNImputer(n_neighbors=5)	Multivariate patterns	credit_score, loan_amount
MICE / IterativeImputer	IterativeImputer()	Complex relationships	multiple cols
Missing Indicator	MissingIndicator()	MNAR — add binary flag	income_was_missing

4. Outlier Detection & Treatment

Definition: An Outlier is a data point that deviates significantly from the overall pattern of the data. Outliers can distort model training and must be detected and treated appropriately.

4.1 Detection Methods

Z-score Method: If |z| = |(x - mean) / std| > 3, the point is an outlier. Assumes normality.

```
from scipy import stats z = stats.zscore(df['annual_income']) outliers = df[abs(z) > 3]
```

IQR Method: IQR = Q3 - Q1. Outliers: x < Q1-1.5*IQR or x > Q3+1.5*IQR. Robust to skew.

```
Q1, Q3 = df['annual_income'].quantile([0.25, 0.75]) IQR = Q3 - Q1 outliers =
df[(df['annual_income'] < Q1-1.5*IQR) | (df['annual_income'] > Q3+1.5*IQR)]
```

Percentile Method: Cap values below p1 and above p99 to eliminate extreme tails.

```
lo, hi = df['annual_income'].quantile([0.01, 0.99]) df['annual_income'] =
df['annual_income'].clip(lo, hi)
```

Winsorization: Replaces extreme values with the nearest non-extreme percentile. Preserves row count.

```
from scipy.stats.mstats import winsorize df['annual_income'] =
winsorize(df['annual_income'], limits=[0.01, 0.01])
```

5. Feature Engineering

Definition: Feature Engineering is the process of using domain knowledge to select, transform, and create new input variables (features) that improve the predictive power of machine learning models.

5.1 Date & Time Features

Date columns are extracted into component features (year, month, day, weekday) that capture temporal patterns.

```
df['join_date'] = pd.to_datetime(df['join_date']) df['join_year'] =
df['join_date'].dt.year df['join_month'] = df['join_date'].dt.month df['join_weekday']
= df['join_date'].dt.dayofweek df['tenure_days'] = (pd.Timestamp('today') -
df['join_date']).dt.days
```

5.2 Categorical Encoding

Method	When to Use	Pros	Cons
Ordinal Encoding	Natural order exists (education)	Preserves order	Assumes linear gaps
Label Encoding	Binary or target column	Simple, compact	Implies false order for multi-class
One-Hot Encoding	Nominal, no order (region, purpose)	No ordinal assumption	Curse of dimensionality
Target Encoding	High cardinality categoricals	Compact, powerful	Risk of data leakage

5.3 Binning & Discretization

Equal-Width Binning: Divides range into equal intervals. Simple but can produce empty bins.

Quantile Binning: Each bin has equal number of records. Robust for skewed distributions.

K-Means Binning: Uses K-Means clustering to find natural bin boundaries in the data.

Binarization: Converts numerical feature to 0/1 based on a threshold (e.g., credit_score > 700).

6. Feature Scaling

Definition: Feature Scaling normalizes the range of independent variables so that no single feature dominates due to its scale. Essential for distance-based and gradient-descent algorithms.

Method	Formula	Range	Best For
Standardization (Z-score)	z = (x - mean) / std	[-inf, +inf]	Normally distributed data, SVM, LR
Min-Max Scaling	x' = (x-min)/(max-min)	[0, 1]	Neural networks, KNN
MaxAbs Scaling	x' = x / max(|x|)	[-1, 1]	Sparse data, already centered
Robust Scaling	x' = (x-median) / IQR	Varies	Data with many outliers

7. Feature Transformations

Definition: Transformations modify the distribution of features to reduce skewness, handle non-linearity, and improve model convergence. They make features more Gaussian-like.

7.1 FunctionTransformer

Log Transform: x' = log(x). Most effective for right-skewed data (income, prices). Compresses large values.

Square Root Transform: x' = sqrt(x). Milder than log; works when data has zeros.

Reciprocal Transform: x' = 1/x. Inverses the distribution; useful for ratio features.

```
from sklearn.preprocessing import FunctionTransformer import numpy as np log_tf =
FunctionTransformer(np.log, validate=True) log_transformed = log_tf.fit_transform(X)
```

7.2 PowerTransformer

Box-Cox Transform: Finds optimal lambda (lambda) to make distribution Gaussian. Requires strictly positive input.

```
from scipy.stats import boxcox bc_data, lambda_ = boxcox(df['loan_amount'] + 1)
print(f'Optimal lambda: {lambda_:.3f}')
```

Yeo-Johnson Transform: Extension of Box-Cox that handles zero and negative values.

```
from sklearn.preprocessing import PowerTransformer pt =
PowerTransformer(method='yeo-johnson') X_transformed = pt.fit_transform(X)
```

7.3 ColumnTransformer

Applies different preprocessing pipelines to different subsets of features in a single, composable step.

```
from sklearn.compose import ColumnTransformer from sklearn.preprocessing import
StandardScaler, OneHotEncoder ct = ColumnTransformer([ ('num', StandardScaler(),
['age','income','loan_amount']), ('cat', OneHotEncoder(drop='first'),
['region','gender']) ]) X_processed = ct.fit_transform(df)
```

8. Key Definitions Glossary

Term	Definition
Data Preprocessing	The set of techniques applied to raw data to make it suitable for ML modelling — including cleaning, imp
Feature Engineering	Creating new or transforming existing features using domain knowledge to improve model performance.
Imputation	The process of replacing missing values with estimated substitutes such as mean, median, mode, or mo
Outlier	A data point that deviates significantly from the rest of the data, potentially indicating noise, error, or rare
Winsorization	Capping extreme values at specified percentiles to reduce the effect of outliers while preserving all data
Ordinal Encoding	Mapping ordered categorical values to integers that preserve their natural rank order.
One-Hot Encoding	Creating a binary column for each category (excluding one to avoid multicollinearity).
StandardScaler	Transforms features to have zero mean and unit variance: z = (x - mean) / std.
RobustScaler	Scales using the median and IQR, making it resistant to outliers.
Box-Cox Transform	A power transformation that makes data more normally distributed, parameterized by lambda.
Yeo-Johnson	A Box-Cox extension that handles zero and negative values in the input.
KNN Imputer	Imputes missing values using the K nearest neighbours based on Euclidean distance across other featu
MICE / IterativeImputer	Models each feature with missing values as a function of other features, iterating until convergence.
Binarization	Converting a numerical feature to 0 or 1 based on a threshold value.
K-Means Binning	Using K-Means clustering to discover natural boundaries for discretizing continuous features.
Debt-to-Income Ratio	Engineered feature = loan_amount / annual_income. High values indicate default risk.
Tensor	A multi-dimensional array. Rank-0 = scalar, Rank-1 = vector, Rank-2 = matrix, Rank-3+ = tensor.
CRISP-DM	Cross-Industry Standard Process for Data Mining — a 6-phase lifecycle for data science projects.